MASSIVELY SCALABLE APPS LAB

Lab 1: Introduction to Spring Boot

PREPARED BY: MOHAMED AYMAN

COURSE ROADMAP

- **Phase 1: Building Services**
  - Spring Boot fundamentals
  - REST APIs + layers
  - Version Control
- **Phase 2: Running Services**
  - Docker
  - Databases
  - Caching
- **Phase 3: Scaling and Monitoring Services**
  - Design Patterns
  - Micro-services
  - Kubernetes
  - Logging and Monitoring
  - Testing

COURSE INFO

Course E-mail	scalable2025@gmail.com	
Dr. Mervat Abu Elkheir	mervat.abuelkheir@guc.edu.eg	C7
Mennaallah Khalifa	mennaallah.khalifa@guc.edu.eg	C1.213
Mohamed Ayman	mohamed.abu-elmagd@guc.edu.eg	C1.213
Dalia Waleed	dalia.walid@guc.edu.eg	C1.213
Mohamed Zaki	mohamed.ahmed-zaki@guc.edu.eg	C7.220
Seif Zidan	seifeldin.zeidan@guc.edu.eg	C1.219
Mohamed Medhat	mohamed.medhat-adel@guc.edu.eg	C1.213
Aml Yasser	aml.yasser@guc.edu.eg	C1.213
Mohamed Amr	mohamed.amr-omar@guc.edu.eg	C1.211

COURSE INFO

Course Weights:

Lab Tasks (Best n-1)	30%
Milestone 1	15%
Milestone 2	15%
Milestone 3 + Final Presentation	40%

Office Hours:

- **Timings and reservation will be done via email and reservation form.**

COURSE INFO

Course guidelines:

- **Attendance**
- **Project Teams**
- **Project Ideas**
- **Course Calendar**
- **Important! Refer to the guidelines pdf posted on the CMS**

SCALABLE APPLICATIONS

REAL SCENARIO (GRADES POSTED ON TRANSCRIPT)

- **8:00 PM:**
  - Transcript Opens.
  - Hundreds of students attempt to get their grades simultaneously.
- **8:01 PM:**
  - Transcript Crashes.
  - No One can access their grades.

REAL SCENARIO (GRADES POSTED ON TRANSCRIPT)

- **Failures happen fast due to:**
  - slow system.
  - errors.
  - duplicate requests.
- **What caused it?**
  - one server + one database + no caching + no scaling plan

REAL SCENARIO (GRADES POSTED ON TRANSCRIPT)

- **Problem**
  - Apps work fine with 10 users…
  - But break badly at 10,000+ users

- **This course teaches you how to build systems that:**
  - don't crash under load
  - scale without rewriting everything
  - remain maintainable + deployable

WHAT "MASSIVELY SCALED" ACTUALLY MEANS

- **Massively Scaled Application =**
  - A system that can handle **huge spikes** safely by scaling resources and managing failure.
- **4 keywords for Massively Scaled Apps:**
  - **Throughput:** how many requests/sec
  - **Latency:** how fast each request responds
  - **Availability:** how often system stays up
  - **Cost:** scaling without burning money

MSAS: EXAMPLES

Google

- As of December 2024, Google was the most visited website globally, with **132.3 billion monthly visits**, averaging about **51,000 visits per second**.

Facebook

- Hosts over 50 billion photos uploaded by users.
- As of 2017, reached 2 billion monthly active users.

Amazon

- Amazon processes approximately **18.5 orders per second**, totaling about **1.6 million packages shipped daily**.

Netflix

- In the first half of 2023, users worldwide watched over **92 billion hours of content**, averaging about **10.5 million hours per hour**.

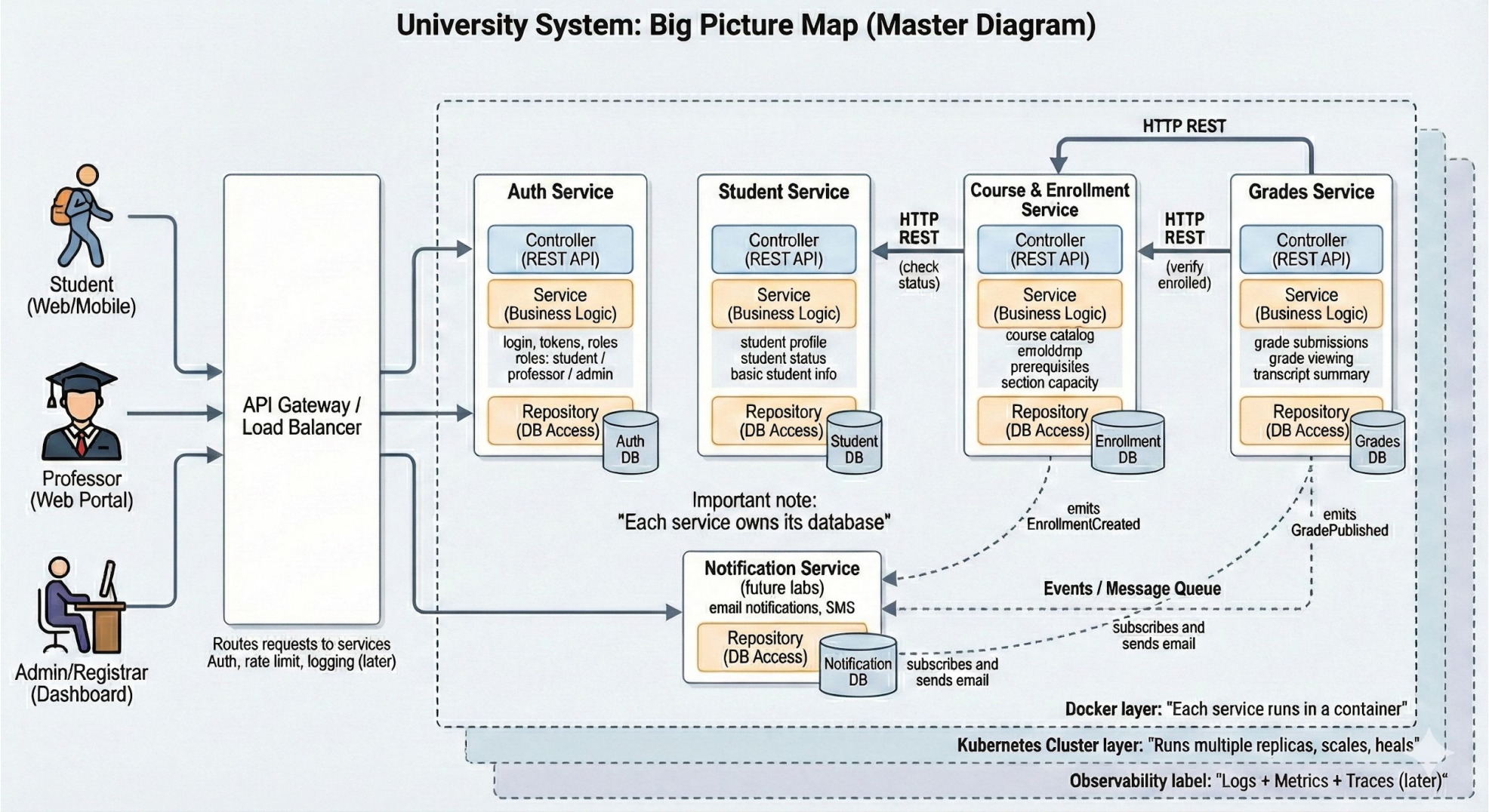
Twitter(Or X)

- On average, around **6,000 tweets** are sent per second.
- The record for tweets per second was set on August 3, 2013, with **143,199 tweets** in a single second.

WHAT USUALLY BREAKS FIRST WHEN YOU SCALE

- **The first bottlenecks are usually:**
  - Database becomes slow (too many reads/writes)
  - Single instance hits CPU/RAM limit
  - Too many open connections / threads
- **Note:**
  - Scaling is not "add server" only, it is **design**.

MSA: UNIVERSITY SYSTEM



WHY THIS MAP MATTERS?

- **Every lab will improve part of this diagram:**
  - **Today:** build one Spring Boot service correctly
  - **Upcoming labs:**
    - Dockerize it,
    - add database,
    - add caching,
    - split into micro-services,
    - deploy using Kubernetes,
    - automate deployment with CI/CD.
  - **Note:**
    - If you understand this map, you will never feel lost.

JAVA AND MAVEN

WHAT IS MAVEN?

- Maven is a **build automation** and **project management** tool used primarily in **Java** projects.
- It simplifies project builds, dependency management, and documentation through a standardized approach.

ROLE IN JAVA

1. **Dependency Management**
   1. Automatically downloads required libraries (JAR files) from repositories like **Maven Central**.
   2. Manages versioning and conflicts between dependencies.
2. **Build Automation**
   1. Compiles Java code, packages it into JAR/WAR files, and runs tests automatically.
   2. Reduces manual effort in managing builds.
3. **Project Standardization**
   1. Follows a structured directory layout, making it easier for teams to collaborate.
   2. Uses a centralized configuration file (**pom.xml**).

EXAMPLE OF A MAVEN FILE

The required package that we will use is between these 2 headers

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
<!--    <dependency>-->
<!--        <groupId>com.google.code.gson</groupId>-->
<!--        <artifactId>gson</artifactId>-->
<!--        <version>2.8.7</version>-->
<!--    </dependency>-->
</dependencies>

<build>
<plugins>
    <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
</plugins>
</build>
```

What to use when we want to build the project

COMMON MAVEN COMMANDS:

Command	Description
mvn install “package”	Installs the project into the local repository.
mvn package	Builds the project and creates a JAR/WAR file.
mvn clean	Deletes target directory (removes compiled files).
mvn compile	Compiles Java source files.
mvn test	Runs unit tests.
mvn dependency:tree	Shows dependency hierarchy.

SPRING BOOT

WHAT IS SPRING BOOT?

- Spring Boot is a **framework** built on top of **Spring Framework** that simplifies Java application development. It allows developers to create **standalone**, **production-ready**, and **micro-services-based** applications with minimal configuration.

KEY FEATURES OF SPRING BOOT

1. **Auto-Configuration**
   - Spring Boot **automatically configures** required components based on the dependencies present in **pom.xml** or **build.gradle**
   - Example: If **spring-boot-starter-web** is added, Spring Boot auto-configures Tomcat and sets up a web application.
2. **Embedded Server**
   - Comes with **Tomcat**, **Jetty**, or **Undertow**, so no need to manually set up a server.
   - Applications can run as **JAR files**, making deployment easier.

SPRING INITIALIZER

spring initializr

Project

- ○ Gradle - Groovy
- ○ Gradle - Kotlin
- ● Maven

Language

- ● Java
- ○ Kotlin
- ○ Groovy

Spring Boot

- ○ 4.1.0 (SNAPSHOT)
- ○ 4.1.0 (M1)
- ○ 4.0.3 (SNAPSHOT)
- ● 4.0.2
- ○ 3.5.11 (SNAPSHOT)
- ○ 3.5.10

Project Metadata

Group	com.example	Artifact	demo
Name	demo		
Description	Demo project for Spring Boot		
Package name	com.example.demo		
Packaging	● Jar ○ War		
Configuration	● Properties ○ YAML		
Java	● 25 ○ 21 ○ 17		

Dependencies

ADD DEPENDENCIES... ⌘ + B

No dependency selected

GENERATE ⌘ + ↵ | EXPLORE CTRL + SPACE | ...

SPRING INITIALIZER

spring initializr

Project
- ○ Gradle - Groovy
- ○ Gradle - Kotlin
- ● Maven

Language
- ● Java
- ○ Kotlin
- ○ Groovy

Spring Boot
- ○ 4.1.0 (SNAPSHOT)
- ○ 4.1.0 (M1)
- ○ 4.0.3 (SNAPSHOT)
- ● 4.0.2
- ○ 3.5.11 (SNAPSHOT)
- ○ 3.5.10

Project Metadata

Group	com.example	Artifact	demo
Name	demo		
Description	Demo project for Spring Boot		
Package name	com.example.demo		
Packaging	● Jar ○ War		
Configuration	● Properties ○ YAML		
Java	● 25 ○ 21 ○ 17		

Dependencies

ADD DEPENDENCIES... ⌘ + B

No dependency selected

Spring Web WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Data JPA SQL
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

H2 Database SQL
Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.

PostgreSQL Driver SQL
A JDBC and R2DBC driver that allows Java programs to connect to a PostgreSQL database using standard, database independent Java code.

Spring Boot DevTools DEVELOPER TOOLS
Provides fast application restarts, LiveReload, and configurations for enhanced development experience.

Rest Repositories WEB
Exposing Spring Data repositories over REST via Spring Data REST.

GENERATE ⌘ + ↵ | EXPLORE CTRL + SPACE | ...

PROJECT STRUCTURE

HOW TO MAP IT TO SPRING BOOT?

1. **Controller:** Handles HTTP requests
2. **Service:** Business logic
3. **Repository:** Data access
4. **Model:** Data structure

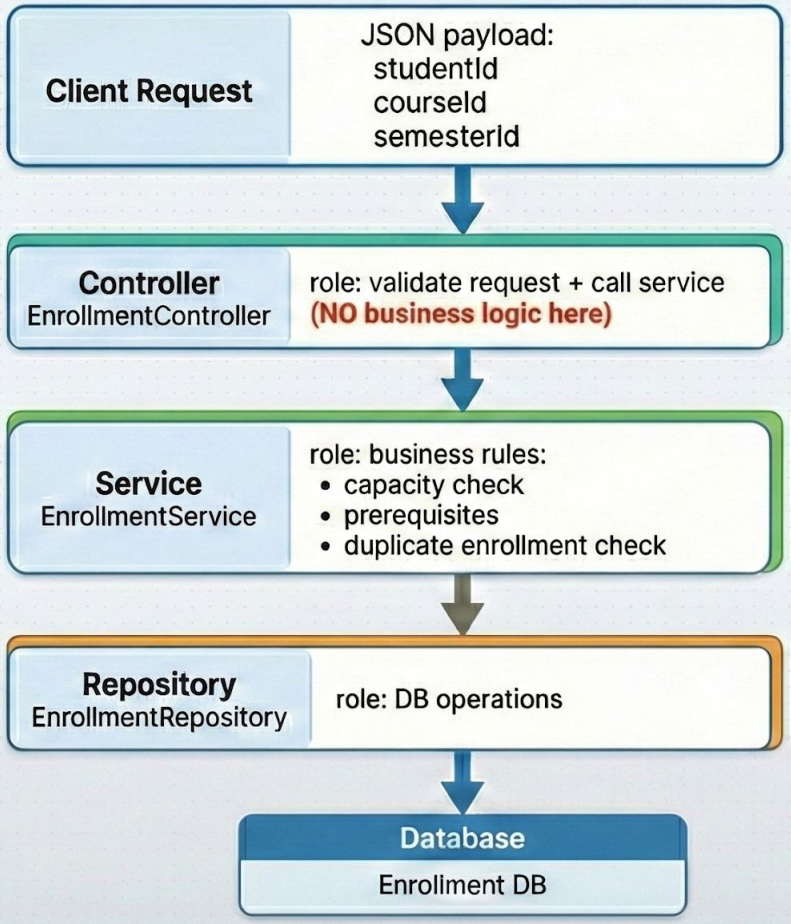
```
src/main/java/com/example/demo
 ├── controller/
 │   ├── UserController.java
 ├── service/
 │   ├── UserService.java
 ├── repository/
 │   ├── UserRepository.java
 ├── model/
 │   ├── User.java
```

INSIDE ONE SERVICE

Request Flow Inside One Service (Spring Boot Anatomy)

POST /enrollments

Enroll student in a course



Where Dependency Injection fits

Controller depends on Service

↓

Service depends on Repository

Spring injects them
(@Autowired or constructor)

LAYERING RULES

Controller (API Layer)

- Handles HTTP + request/response only
- Calls the service

Service (Business Layer)

- Rules + validations + logic
- Orchestrates workflows

Repository (Data Layer)

- Database operations only

- **Why This matters?**
  - Easier testing
  - Easier scaling
  - Cleaner micro-services later

ANNOTATIONS

WHAT ARE ANNOTATIONS?

- Annotations in Java are **metadata** used to provide additional information to the compiler, runtime, or frameworks like **Spring Boot**.

- Spring Boot **heavily relies on annotations** for configuration, dependency injection, and defining REST APIs.

COMMONLY USED SPRING BOOT ANNOTATIONS

1. @SpringBootApplication

- This annotation **marks the main class** of a Spring Boot application and enables:
  - ✔ Auto-Configuration
  - ✔ Component Scanning
  - ✔ Spring Boot Configuration

```
@SpringBootApplication
public class MySpringBootApplication {
    public static void main(String[] args) {
        SpringApplication.run(MySpringBootApplication.class, args);
    }
}
```

COMMONLY USED SPRING BOOT ANNOTATIONS

2. @RestController

- Marks a class as a **RESTful web service** controller.
- It combines **@Controller** and **@ResponseBody**, meaning returned values are converted to JSON.

```
@RestController
@RequestMapping("/api")
public class MyController {

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello, Spring Boot!";
    }
}
```

COMMONLY USED SPRING BOOT ANNOTATIONS

3. @GetMapping & @PostMapping

- These annotations **handle HTTP requests** in REST APIs.
  - **@GetMapping** → Handles HTTP GET requests.
  - **@PostMapping** → Handles HTTP POST requests.

```java
@RestController
@RequestMapping("/users")
public class UserController {

    @GetMapping("/{id}")
    public String getUser(@PathVariable int id) {
        return "User ID: " + id;
    }

    @PostMapping("/add")
    public String addUser(@RequestBody String user) {
        return "User added: " + user;
    }
}
```

COMMONLY USED SPRING BOOT ANNOTATIONS

4. @Component, @Service, @Repository

- These annotations **define Spring-managed beans** for different layers of the application.
  - **@Component** → Generic Spring bean (can be used anywhere).
  - **@Service** → Used for business logic layer.
  - **@Repository** → Used for data access layer (DAO) and exception handling.

```java
@Component
public class MyComponent {
    public String message() {
        return "Hello from Component!";
    }
}
```

```java
@Service
public class MyService {
    public String process() {
        return "Processing business logic";
    }
}
```

```java
@Repository
public class MyRepository {
    public String fetchData() {
        return "Fetching data from database";
    }
}
```

CONTROLLER

WHAT WILL HAPPEN IF WE DID THIS?

```java
@SpringBootApplication
public class DemoApplication {

    @GetMapping //to make it a get request
    public String Hello(){
        return "Hello World!!";
    }

    @GetMapping //to make it a get request
    public List<Student> getStudents(){
        return studentService.getStudents();
    }

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

WHAT WOULD HAPPEN IF WE DID THIS?

```java
@SpringBootApplication
public class DemoApplication {

    @GetMapping //to make it a get request
    public String Hello(){
        return "Hello World!!";
    }

    @GetMapping //to make it a get request
    public List<Student> getStudents(){
        return studentService.getStudents();
    }

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

It doesn't know how to map different request!!

```
org.springframework.beans.factory.BeanCreationException: Error creating bean with name 'requestMappingHandlerMapping' defined in class path resource [org/springframework/boot/auto
com.example.demo.DemoApplication#getStudents()
to {GET [/ || ]}: There is already 'demoApplication' bean method
com.example.demo.DemoApplication#hello() mapped.
    at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.initializeBean(AbstractAutowireCapableBeanFactory.java:1808) ~[spring-beans-6.2.1.jar:6.2.1]
    at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.doCreateBean(AbstractAutowireCapableBeanFactory.java:601) ~[spring-beans-6.2.1.jar:6.2.1]
    at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.createBean(AbstractAutowireCapableBeanFactory.java:523) ~[spring-beans-6.2.1.jar:6.2.1]
    at org.springframework.beans.factory.support.AbstractBeanFactory.lambda$doGetBean$0(AbstractBeanFactory.java:336) ~[spring-beans-6.2.1.jar:6.2.1]
    at org.springframework.beans.factory.support.DefaultSingletonBeanRegistry.getSingleton(DefaultSingletonBeanRegistry.java:289) ~[spring-beans-6.2.1.jar:6.2.1]
    at org.springframework.beans.factory.support.AbstractBeanFactory.doGetBean(AbstractBeanFactory.java:334) ~[spring-beans-6.2.1.jar:6.2.1]
    at org.springframework.beans.factory.support.AbstractBeanFactory.getBean(AbstractBeanFactory.java:199) ~[spring-beans-6.2.1.jar:6.2.1]
    at org.springframework.beans.factory.support.DefaultListableBeanFactory.instantiateSingleton(DefaultListableBeanFactory.java:1122) ~[spring-beans-6.2.1.jar:6.2.1]
    at org.springframework.beans.factory.support.DefaultListableBeanFactory.preInstantiateSingleton(DefaultListableBeanFactory.java:1093) ~[spring-beans-6.2.1.jar:6.2.1]
    at org.springframework.beans.factory.support.DefaultListableBeanFactory.preInstantiateSingletons(DefaultListableBeanFactory.java:1030) ~[spring-beans-6.2.1.jar:6.2.1]
    at org.springframework.context.support.AbstractApplicationContext.finishBeanFactoryInitialization(AbstractApplicationContext.java:987) ~[spring-context-6.2.1.jar:6.2.1]
```

THE NEED FOR THE CONTROLLER

- Hence, here the initial
- The Controller handles all the get and post requests as it is responsible for the API callings
- Hence, we use the annotation:
  - **@RestController**

DEPENDENCY INJECTION

Let's Take a step back:

- When you need something, do you…
  - Build it yourself??
  - Ask Someone to provide it for you?
- Another Examples:
  - When you want bread — do you grow wheat or go to a bakery?
  - When you need a car — do you build one or buy one?
  - When you want a phone — do you manufacture it or get it from a store?

Most of the time, we **ask for things we need** rather than creating them ourselves!

Let’s Catch the Pattern:

Who Needs	What They Need	Who Provides
👨‍🍳 Chef	Ingredients	Kitchen Manager
🎬 Director	Actors	Casting Agency
🏥 Surgeon	Medical Equipment	Hospital
👩‍🏫 Teacher	Textbooks	School Administration
✈️ Pilot	Aircraft	Airline Company

"Don't create what you need — receive what you need!"

So what is Dependency Injection?

Dependency Injection (DI)

Let's break down the term:

📦 Dependency

Something that a class **needs** to do its job

💉 Injection

The act of **providing** that dependency from outside

Dependency Injection = Providing an object's dependencies from the outside, rather than having the object create them itself.

In terms of Components:

Let's build a student management system with three layers:

StudentController
Handles HTTP requests

needs →

StudentService
Business logic

needs →

StudentRepository
Database access

🤔 How should Controller get the Service it needs?

❌ What we don't do

```java
@RestController
public class StudentController {

    // Creating the dependency ourselves - BAD!
    private StudentService service = new StudentService();

    @GetMapping("/students")
    public List<Student> getStudents() {
        return service.getAllStudents();
    }

public class StudentService {
    // Service creates its own Repository - BAD!
    private StudentRepository repo = new StudentRepository();
}

public class StudentRepository {
    // Repository creates its own connection - BAD!
    private DatabaseConnection db = new DatabaseConnection();
}
```

✅ Instead we do this:

```java
@RestController
public class StudentController {

    private final StudentService service;  // Declare what you need

    // Receive it through constructor - GOOD!
    public StudentController(StudentService service) {
        this.service = service;
    }

    @GetMapping("/students")
    public List&lt;Student&gt; getStudents() {
        return service.getAllStudents();
    }
}
```

How this happens?

When your application starts, Spring:

1. **Scans:** your code for classes marked with annotations
2. **Finds**: @Component, @Service, @Repository, @Controller
3. **Creates:** instances of these classes (called "Beans")
4. **Checks:** each constructor to see what dependencies are needed
5. **Injects:** the required beans automatically!

✅ Complete Working Example:

```java
// 1. Repository Layer
@Repository
public class StudentRepository {
    public List&lt;Student&gt; findAll() { /* DB query */ }
}

// 2. Service Layer
@Service
public class StudentService {
    private final StudentRepository repository;

    public StudentService(StudentRepository repository) {
        this.repository = repository;  // Injected!
    }

    public List&lt;Student&gt; getAllStudents() {
        return repository.findAll();
    }
}

// 3. Controller Layer
@RestController
public class StudentController {
    private final StudentService service;

    public StudentController(StudentService service) {
        this.service = service;  // Injected!
    }
}
```

Common Mistakes:

Error: "No beans of type 'StudentService' found"

- **This happens when:**
  - You forgot to add **@Service** annotation to your service class
  - The class is not in a package that Spring scans
  - There's a typo in the class name

CRUD

WHAT IS CRUD?

- CRUD stands for **Create, Read, Update, and Delete** – the four fundamental operations for managing data in applications.
- **Create (C)** → Adds new data.
- **Read (R)** → Retrieves existing data.
- **Update (U)** → Modifies existing data.
- **Delete (D)** → Removes data.

CREATE (C): POST

```
@PostMapping //to make it a post request
public void addStudent(@RequestBody Student student){

    studentService.addStudent(student);
}
```

READ (R): GET

```
@GetMapping //to make it a get request
public List<Student> getStudents(){
  return studentService.getStudents();
}
```

UPDATE (U): PUT

```
@PutMapping //to make it a put request
public void putStudent(@RequestBody Student student){

    studentService.updateStudent(student);
}
```

DELETE (D): DELETE

```
@DeleteMapping //to make it a delete request
public void deleteStudent(@RequestBody String id){

  studentService.deleteStudent(id);
}
```

LAB 1: SUMMARY

- Generate and run a spring boot project
- Build a simple REST API
- Apply correct structure:
  - Controller → Service → Repository
- Understand DI (dependency injection)

REFERENCES FOR EXTRA KNOWLEDGE

- https://youtu.be/GATSXm7WAxU?si=1PLrptAeUfEt-rAd
- https://youtu.be/J1f5b4vcxCQ?si=Oa3XwWuXWAZRCF-X
- https://youtu.be/9SGDpanrc8U?si=xFkZFrmz3KV8p0by

THANK YOU